Rollins Tennis Archive – Java Full Stack Application

CMS 270 – Object Oriented Programming

Instructor:

Dr. Sirazum Munira Tisha

Team Members & Roles:

Madhav Khanal: Lead Programmer

Stella Fruijtier: Data Scientist, Data Visualization

Richard Stoiberer: Project Manager

December 2, 2025

Rollins Tennis Archive – Java Full Stack Application

1. Introduction:

Rollins Tennis Archive is a full-stack web application built with a Java REST API backend and a React frontend interface. The system stores and displays player information and match results from the past three Rollins College tennis seasons for a selected number of players and presenting the data through a clean and interactive dashboard. Users can easily browse player bios, review match details, and analyze performance trends across seasons.

The primary purpose of the application is to organize historical team data in one accessible location and make it easier for users to review past performances than on the current Rollins Tennis website. By supporting filtering, searching, and statistics generation, the system helps coaches, players, and fans quickly find relevant results and understand performance patterns. The project also aims to demonstrate how a real-world sports dataset can be managed using proper object-oriented design and a structured backend-frontend architecture, potentially being a guiding project for our team to select our Computer Science Capstone Project.

Our team selected this topic because of our close connection to the Rollins College tennis team, as some members of our team compete on the varsity tennis program, and our shared interest in sports and data. We recognized that tennis results were often scattered or difficult to revisit, and we wanted to create a tool that makes past performances easy to access and analyze. At the same time, tennis naturally provides a clear domain for applying OOP principles-such as modeling players, matches, seasons, and results-which made it an ideal fit for a Java OOP project.

2. UML Diagram

Figure 1 shows the complete UML class diagram for the Rollins Tennis Archive backend. The diagram models all major components of the system, including classes, abstract classes, and enumerations. It highlights how the application uses object-oriented design to represent players, matches, seasons, and statistical logic in a structured way.

At the core of the diagram is the abstrac *Match* class, which defines the shared attributes and behaviors of all tennis matches. *Match* stores the match date, season, opponent, score/result, and provides methods such as *getWinner(), getParticipants(),* and *getMatchType().*

Two subclasses extend this abstract class:

- *SinglesMatch* - Contains one Rollins player and one opponent player.
- *DoublesMatch* - Contains a pair of Rollins players and a pair of opponents.

Both subclasses override shared behaviors while providing their own implementations of *getParticipants()* and *getMatchType().*

The *Player* class stores detailed player information (ID, name, nationality, class year, and UTR(Universal Tennis Rating)), and is associated with both match subclasses through composition. Each *Match* also contains a *Result* object that stores the score and whether Rollins won as a team, demonstrating a “has-a” relationship.

The *Season* enumeration provides the set of valid seasons (2022–2024) and is used throughout the model to categorize matches.

MatchRepository is responsible for storing all Match objects and organizes them via a *Map<Season, List<Match>>*, allowing fast access to matches by season. It offers methods such as:

- *addMatch(Match match)*

- *getMatchesBySeason(Season season)*
- *getWinCount()*

RosterManager maintains all *Player* objects using a *List<Player>* and a secondary lookup map. It provides methods to add new players, retrieve players by ID, and return the full roster. *StatsService* sits above the repository layer and computes aggregated statistics such as:

- Win/loss records by player
- Season records
- Individual win percentage

This class demonstrates high cohesion and clean separation of concern by isolating all statistical logic away from the controller.

The *TennisController* connects the backend logic to the external world. It uses dependency references to *RosterManager*, *MatchRepository*, and *StatsService* to expose the main API endpoints. These endpoints return lists of players, match data, or computed statistics, which the React frontend uses to render the application interface.

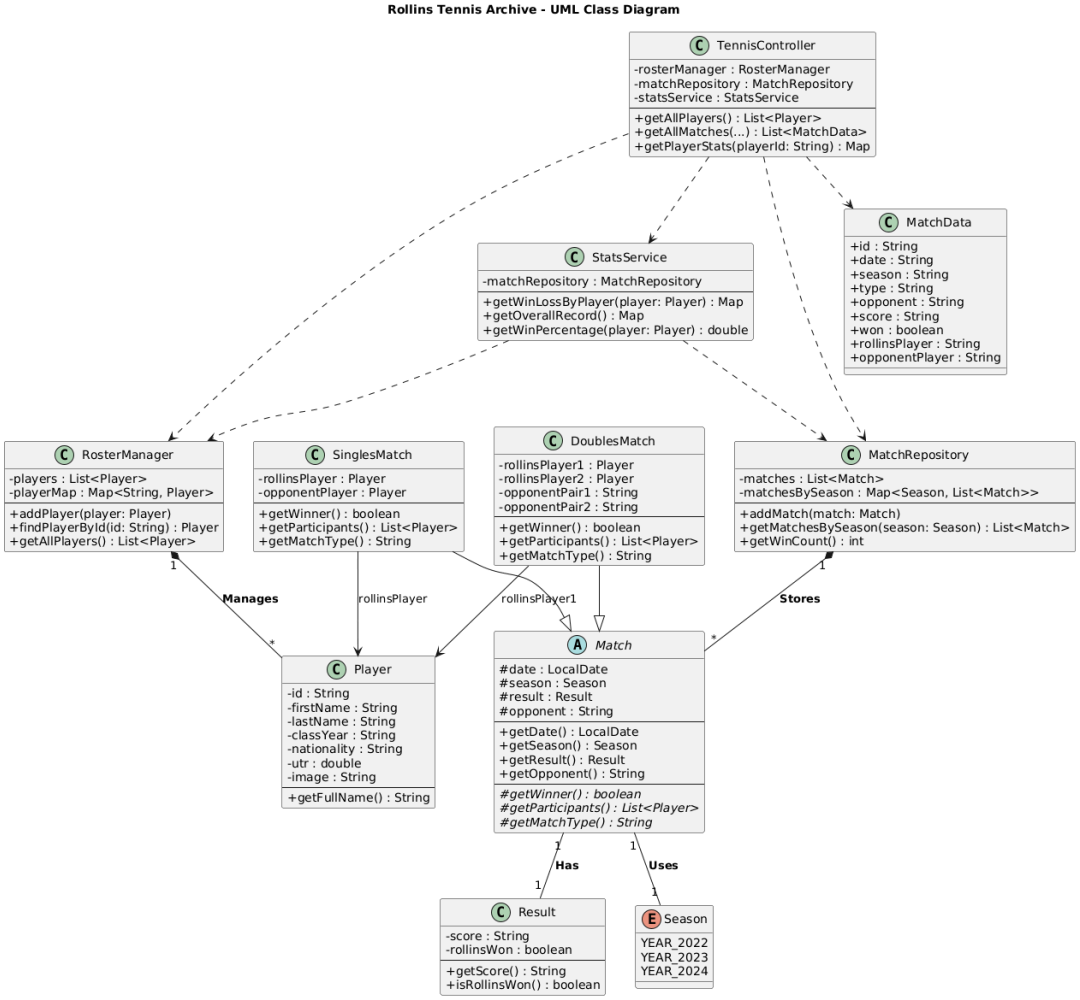

Figure 1

3. Implementation Description

The Rollins Tennis Archive application was designed to demonstrate strong Object-Oriented Programming practices while modeling a realistic sports data system. This section explains how core OOP principles, including abstraction, encapsulation, inheritance, and polymorphism, were applied throughout the implementation.

3.1. Classes and Objects

We structured the system around real-world tennis entities, which translate into Java classes. The domain model includes *Player, Match, SinglesMatch, DoublesMatch, Result,* and *Season*. Each object represents a meaningful unit in the tennis domain.

- A player object holds attributes such as name, nationality, class year, and UTR rating.
- A Match (and its subclasses) stores the match date, opponent information, season, and final score.
- A Result object encapsulates the scoring details and win/loss outcome.
- A Season enum ensures that only valid season values (2022-2024) can be used.

These classes form the foundation of the system, allowing the backend to model tennis data in a structured and consistent manner.

3.2. Encapsulation

Encapsulation is used extensively across the project. All class fields are declared *private*, preventing external classes from modifying internal states directly. Instead, access is controlled through getter and setter methods.

Example: In the *Player* class, fields such as *id, firstName, lastName*, and *utr* are private. Other components can retrieve this information only through methods like *getFullName()* or *getUtr()*. This approach protects the integrity of player data and makes the code easier to maintain.

Encapsulation also ensures clean separation between the internal logic of data classes and the external services that depend on them.

3.3. Inheritance

Inheritance is demonstrated through the *Match* class hierarchy. *Match* is an abstract superclass that defines shared fields and methods for all match types, including:

- *date*
- *season*
- *result*
- *opponent*
- *getWinner()*
- *getMatchType()*
- *getParticipants()*

Two concrete subclasses inherit from this abstract class:

- *SinglesMatch* – represents a singles match (one Rollins player vs. one opponent).
- *DoublesMatch* – represents a doubles match (two Rollins players vs. two opponents).

Both subclasses extend the core functionality of *Match* while adding fields unique to their format. This avoids code duplication and keeps all shared logic defined in one place.

3.4. Abstraction

The *Match* class itself is abstract, meaning it cannot be instantiated on its own. It defines the shared structure and behavior of all matches but leaves the implementation of certain methods (such as *getParticipants()* or *getMatchType()*) to the subclasses.

This abstraction allows the system to treat all matches uniformly while still respecting the differences between singles and doubles.

Another example of abstraction is the *StatsService* class. It exposes high-level methods like:

- *getWinLossByPlayer(Player player)*
- *getOverallRecord()*
- *getWinPercentage(Player player)*

These methods hide the underlying data processing logic, making statistics retrieval simple for the REST controller and frontend.

3.5. Polymorphism

Polymorphism plays an important role in how the system manages matches. Because both *SinglesMatch* and *DoublesMatch* inherit from *Match*, the application can store and process them in the same collections.

Example: *MatchRepository* maintains a *List<Match>* and a *Map<Season>, List<Match>>*. When filtering or counting matches, the repository does not need to know whether each match is singles or doubles. It simply calls polymorphic methods like *getWinner()* or *getParticipants(),* and the subclass implementation is used automatically.

The same polymorphism applies in the statistics logic: *StatsService* loops over *Match* objects and evaluates them using shared methods without caring about the specific subclass.

3.6. Data Structures and Services

We use appropriate data structures to organize and retrieve information efficiently:

- *RosterManager* maintains all players using a *List<Player>* and a lookup *Map<String, Player>* for fast ID-based access
- *MatchRepository* stores all matches in both a master list and a map grouped by season
- *StatsService* computes aggregated statistics by iterating over these structures.

This layered design keeps the logic modular and easy to extend. For example, adding new seasons or match types requires minimal changes.

3.7. Backend-Frontend Integration

The system follows a clean separation between backend logic and frontend display. *TennisController* exposes REST API endpoints that serialize Java objects into JSON using Gson. The React frontend retrieves this data using api.js and displays it in:

- PlayersTab
- MatchesTab
- StatsTab

Because the backend objects are designed with clear getters and a consistent structure, the frontend can process them easily.

4. Challenges and Solutions

During the development of the Rollins Tennis Archive application, our team encountered several technical and design-related challenges. These issues helped create the final structure of the system and overcoming them increased our understanding of full-stack development and OOP design. Below are the most relevant challenges we faced and how we addressed them.

4.1. Designing an OOP structure that works for both singles and doubles

Challenge: In the early stages, we struggled to create a class hierarchy that could accurately model both singles and doubles matches without duplicating code. Both formats have different numbers of players and different participant structures, which made the initial design messy and inconsistent.

Solution: We implemented an abstract *Match* class to hold shared fields, such as date, season, opponent, score, and result, and created two subclasses (*SinglesMatch* and *DoublesMatch*) for the specific formats. This allowed us to keep shared behavior in one place while letting each subclass implement its own version of methods like *getParticipants()* and *getMatchType()*. This cleaned up the codebase and made the match model scalable.

4.2. Organizing match data for efficient filtering and statistics

Challenge: We needed an efficient way to store and retrieve matches by season and match type. Storing everything in one single list made filtering slow and cluttered the code with repeated checks.

Solution: *MatchRepository* now stores all matches in both a master *List<Match>* and a *Map<Season, List<Match>>*. This structure gives us a complete list when needed and fast season-based lookups for statistics and filtering. This approach simplified both the controller and the statistics logic and improved the overall performance of the backend.

4.3.Ensuring consistent data transfer between backend and frontend

Challenge: The React frontend relies on the backend returning clean, well-structured JSON. Any inconsistency in field names or formats caused display issues or incorrect filtering in the UI.

Solution: We standardized all data models and ensured that the Java classes used clear, consistent field names and getters. We also used Gson for serialization, which allowed us to keep responses predictable across all endpoints. Once the backend and frontend agreed on a stable format, the UI became much more reliable.

4.4.Creating meaningful statistics while keeping logic maintainable

Challenge: Implementing win/loss calculations, player statistics, and season summaries required looping through mixed match types and handling both singles and doubles scenarios. Without careful planning, the statistical logic became overly complicated.

Solution: We moved all statistics-related computations into a dedicated *StatsService* class. By centralizing this logic, we avoided code duplication and made the calculations easier to test and expand. Polymorphism also helped, since *StatsService* interacts with Match objects generically and doesn't need to distinguish between subclasses.

5. Contribution summary

Our team collaborated throughout the development of the Rollins Tennis Archive application, sharing responsibilities in design, implementation, testing, and documentation. While all members contributed to debugging, planning, and refining the system architecture, each person also focused on specific areas. The table below summarizes the primary contributions.

Madhav Khanal: Led backend development, including implementing *MatchRepository*, *StatsService*, and major portions of the REST API. Built filtering logic, statistics calculations, and endpoint structures in *TennisController*, and performed backend testing using curl.

Richard Stoiberer: Crafted the domain model (Player, Match, SinglesMatch, DoublesMatch, Result, Season) and created the full UML class diagram. Contributed to backend logic, sample data creation, and API testing. Along with Stella, co-wrote the written project report and helped consolidate the final documentation.

Stella Fruijtier: Developed the React frontend and UI/UX design, including the layout of *PlayerTab*, *MatchesTab*, and *StatsTab*. Implemented search bars, filters, tables, and component styling. Managed frontend-backend integration through *api.js*.

Collaborative work: All members contributed to:
- Planning the system architecture
- Creating and testing sample datasets
- Debugging issues across the stack
- Reviewing and improving each other's work
- Preparing final presentation and demonstration